

The money moved. The procedure didn't finish.

A computer-use automation layer where replay guarantees effects, not steps.

interface.ai builds AI agents for roughly a hundred US community banks and credit unions. Their agents talk to members across voice and chat. What those agents cannot do is *finish the job*, because finishing means operating four vendors' legacy consoles that will never expose APIs to one another. This document sets out the specific problem that creates, and the system we propose to build for it.

1. Where the expensive work actually lives

The assignment's own examples are single-app: look up a member, read a balance. That is a useful building block. It is not where the cost is.

The expensive work in a bank back office is **cross-system**. The reason there is no API is not only that the core banking platform is old. It is that the procedure spans four separate vendors with no commercial reason to integrate with each other. A human is the integration layer.

The clearest example, and the one we propose to build against, is **Regulation E dispute processing**. A member reports a card transaction they did not make. Here is everything that has to happen next.

Step	System	Type
Verify member, pull the transaction	Core banking (Symitar / DNA / Keystone)	read
Check for a prior or duplicate dispute	Core + case tracker	read
File the chargeback	Card processor portal (CO-OP / PSCU / Visa DPS)	irreversible
Post provisional credit	Core banking	moves real money
Send the required member notice	Document system	regulated deadline
On resolution: finalise or reverse + notice	Core + documents	irreversible

Four systems. No API between them. At most community institutions this runs on a spreadsheet and a shared inbox.

The regulator makes it unforgiving

Under 12 CFR 1005.11, an institution generally has **10 business days** to investigate a notice of error, or it must post provisional credit within those 10 business days and may then take up to **45 calendar days**. It must notify the member of that credit within **2 business days**. For certain new accounts, point-of-sale and foreign-initiated transactions the window extends to 90 days.

Critically: **missing the deadline is itself a violation**, independent of whether the transaction turned out to be unauthorised. The clock does not care who was right.

2. The three failures that survive normal verification

Every record-and-replay system verifies that a control was actuated and a page was reached. When the action moves money, three failures pass that check cleanly.

1. Double application

A step fails partway through a procedure. The retry starts again from the top. Provisional credit is posted a second time and the member is paid twice. Nothing in the run looks wrong.

2. Wrong entity

The locator resolves correctly, on the wrong record. Member 8830142 and 8830124 both appear in a fuzzy search, in non-deterministic row order. The click lands, the checkpoint passes, the confirmation screen loads, **SUCCESS** is returned, and the wrong member was credited. A checkpoint asserting "the member ID appears somewhere on this page" passes for both.

3. Orphaned procedure

Provisional credit posts. The chargeback filing times out at the processor portal. The 2-business-day notice never goes out. The process died between two systems and nothing durable knew where it was. Three weeks later a compliance officer finds it.

THE PROBLEM, STATED ONCE

The money moved, the procedure didn't finish, and nobody knows. And it cannot simply be retried, because a retry pays the member twice.

3. The solution: Tandem

A capability is not a recorded flow. It is a recorded flow **plus a declared effect on the world**, and the replay engine's job is to guarantee that effect happens **exactly once**.

The system uses an LLM to discover how to drive a legacy UI the first time, then compiles that run into a typed, versioned artifact that replays with no model in the decision loop. That much the assignment requires. The four decisions below are what make it safe enough to point at money.

Move 1 — Effect-typed capabilities with a precheck

Every capability declares what it does to the world: `READ` changes nothing, `STAGE` is reversible, `COMMIT` is irreversible, moves money, or starts a regulatory clock. Every `COMMIT` must additionally declare how to tell whether it has already happened.

```
"effect": {
  "class": "COMMIT",
  "idempotency_key": "regE:{{input.case_id}}:provisional_credit",
  "precheck": {
    "capability": "core.find_memo_by_ref",
    "params": { "account": "{{input.member_id}}", "ref": "{{input.case_id}}" },
    "if_found": "ALREADY_APPLIED"
  },
  "compensation": "core.reverse_provisional_credit",
  "bounds": { "max_amount": 500.00, "currency": "USD" }
}
```

The engine runs the precheck before it acts. If the credit is already on the account it returns `BUSINESS_OUTCOME / ALREADY_APPLIED` and touches nothing. A `COMMIT` that declares no precheck fails schema validation and cannot be saved. Risk is derived from the effect class, never guessed from whether a button looks dangerous.

Move 2 — A procedure ledger above the capabilities

One append-only record per case, written after every step, crash-safe. It knows what ran, in which system, with what result, what money moved, and what the clock says.

```
case D-8842   member 8830142   $340.00   opened 2026-09-02

[ok]   core.lookup_transaction      READ   ok
[ok]   core.check_prior_dispute     READ   none
[FAIL] processor.file_chargeback     COMMIT RECOVERABLE / SESSION_EXPIRED
[ok]   core.post_provisional_credit  COMMIT ok, memo MC-7741   ** MONEY MOVED **
[.]    docs.send_2day_notice         COMMIT pending -- DUE 2026-09-04 17:00
```

Kill the process anywhere and resume knows exactly what is done, what moved money, and what is overdue. Deadlines are computed in business days where the regulation says business days. This is a state machine and a file, cheap to build, and it is the piece that makes this a banking system rather than a browser toy.

Move 3 — Guards scoped to the control, not the page

An assertion of the form "is this text anywhere on the page" is not an identity check. On a results screen listing five members it passes for all five.

Before any `COMMIT` step fires, the engine reads the account number and the amount **out of the same container as the control it is about to press**, and compares both to the declared inputs. A mismatch is a hard failure and nothing commits. Transposed digits and fat-fingered amounts are the real failure modes in servicing, and both are caught here.

Move 4 — Escalation carries the clock, not just the screen

When the procedure stalls, the request routed to a human operator reads:

Case D-8842. Provisional credit is posted — money has moved. Chargeback filing failed at the processor portal with a session expiry. The 2-business-day member notice is not sent and is due tomorrow at 5pm. Here is the live session at the point of failure.

The operator drives the **same live browser session**, not a fresh one, completes the step, and hands control back. Exactly one party holds control at any moment, enforced by an explicit state machine. The ledger records who did what and when, which is the audit artifact a compliance officer actually needs.

4. Why this is the right shape for interface.ai

Their existing products take the dispute: Agentic Voice, Agentic Chat, Fraud Prevention AI. They state plainly that they are not a core banking replacement and integrate with existing cores. So the member-facing half already exists and sells. The half that is missing is hands that can finish the work inside four legacy consoles, safely enough that a credit union will let it move money unattended.

That is precisely the layer this assignment describes, and Reg E disputes is the use case where the difference between "the replay succeeded" and "the effect happened exactly once" is worth real money to their customers.

5. What gets built, and what gets stubbed

Status	Component
Built for real	A deliberately hostile core-banking simulator (framesets, table layouts, no test IDs, unstable generated element ids) with seeded runtime conditions; a second instance of it as another institution; the surface abstraction; LLM discovery; the compiler; the replay engine; the procedure ledger; the policy layer; the live-session handoff
Deliberately thin	The card processor portal — one screen and one failure switch. Its only job is to fail on cue so the ledger has something to survive
Stubbed at a clean seam	The document/notice system, the case tracker, the operator console UI. The transfer <i>mechanism</i> is real; the interface is bare HTML
Designed, not built	Capability registry, credential vaulting, desktop surface, queues. The abstractions leave room for them; building them is explicitly not rewarded

Two capabilities only, one procedure, one genuine model-driven discovery run with the evidence committed.

6. The demonstration

Seven runs, all reproducible from the README. Three of them carry the argument.

Run	Expected result
Discovery	The model drives the hostile UI and completes a provisional-credit posting. Artifact compiled and committed
Replay, new case	Zero LLM calls. Succeeds, returns the memo code
Replay the same case again	Precheck finds the existing credit. <code>ALREADY_APPLIED</code> . Does not pay the member twice
Replay with a transposed ID	Container guard reads the account number from the panel holding the submit button. <code>ENTITY_BINDING_MISMATCH</code> , expected vs observed. Nothing posted
Kill mid-procedure, resume	Ledger knows money moved, the notice is outstanding, the deadline is tomorrow, and routes to a human with all of it
Handoff	Automation pauses on a compliance interstitial, an operator clears it on the same live page, automation finishes, the ledger records the human's action
Second institution	The same artifact runs against a differently-branded, older build of the same product via an overlay, reporting a drift signal

7. The known weak point

Enterprise dispute-management platforms exist — Quavo, Pegasystems Smart Dispute — and some card processors expose dispute APIs. So the

claim "there is no API for this" is weakest exactly where we have planted the flag, and an interviewer who has sold into this space may know it.

The answer, which belongs in the write-up rather than improvised under questioning: those platforms are priced for institutions far larger than the community banks and credit unions this company serves. That is why the work is still done by hand there, and why an automation layer that drives the existing consoles is the realistic path rather than a procurement project.

The architecture does not depend on the domain. If dispute processing proves to be the wrong beachhead, **fee reversal** is the fallback: a top-ten call reason at every credit union, a teller-screen function with no API by any reading, real money, a natural policy gate on waiver frequency, and the same double-application and wrong-entity problems in a fraction of the build.

In one line. Most systems verify that the click landed. This one verifies that the effect happened, that it happened to the right person, that it happened once, and that whatever had to follow it actually followed.